

训练服务器指导手册

1、服务器登录

1.1、事项说明

由于 GPU 卡资源有限，规定一个队伍只能使用一张卡进行训练，否则取消服务器的使用权限。大家可以选择空闲 GPU 卡或者 Volatile GPU-Util 利用率小于 85%的 GPU 卡进行训练，建议优先使用空闲卡进行训练；

1.2、云安全访问服务登录

- 在访问服务器之前，需要登录云安全访问服务，首先下载一个云安全访问服务客户端，点击[云安全服务器客户端](#)下载即可。下载安装成功后，如下图所示：



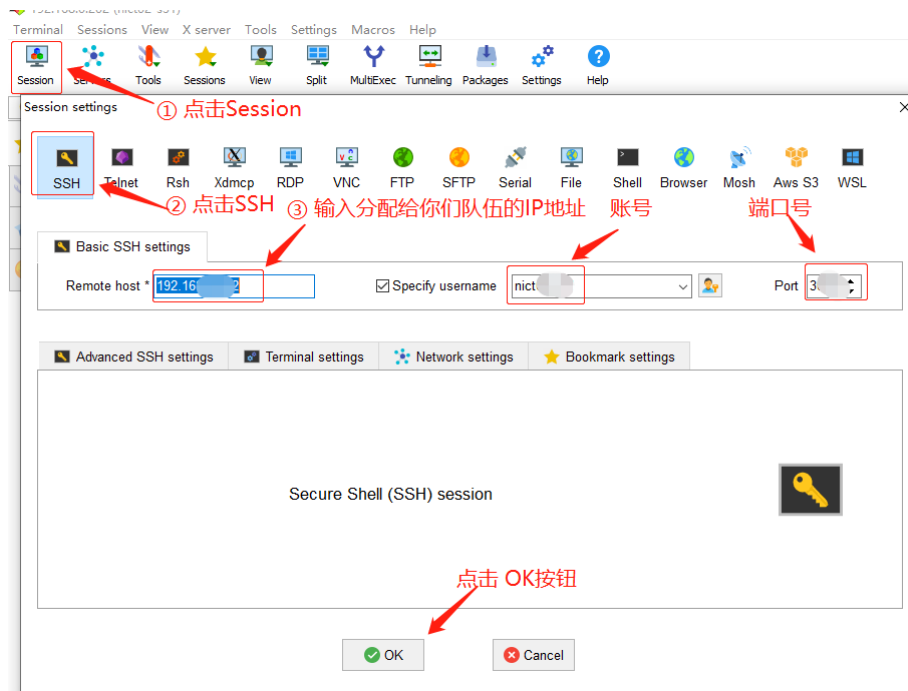
- 一个队伍对应一个 云安全访问服务账号，具体以发布为准，大家找到各自队伍对应的云安全访问服务账号和密码即可，登录成功后，如下图所示：



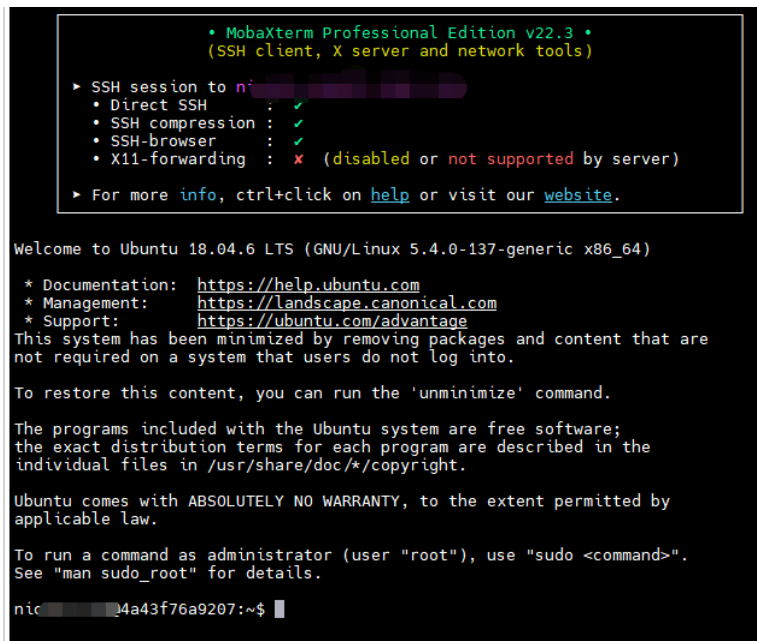
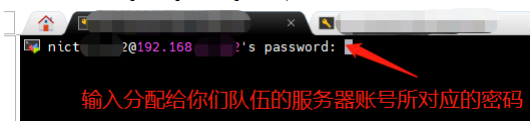
- 若需要退出云安全访问服务客户端，密码为 **Nihao123456!**
- 若需要卸载云安全访问服务客户端，密码为空

1.3、使用SSH登录训练服务器

- 步骤1：下载[MobaXterm工具](#),当然如果你电脑里面有安装其他的ssh工具也是可以的。
- 步骤2：获取你所在队伍的服务器账号、密码、端口号等，这个可以咨询海思老师或者队长。
- 步骤3：使用MobaXterm登录到，队伍所分配的服务器的ssh。



- 步骤4：输入分配给你们队伍的服务器账号所对应的密码，然后敲回车，就能进入到训练服务器的命令行终端了。



- 步骤5：在训练服务器的命令行终端，输入下面的命令，即可看到训练服务器的CUDA版本为11.3

```
nvcc -V
```

```
See man sudo_root for details.
nict@4a43f76a9207:~$ nvcc -V
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2021 NVIDIA Corporation
Built on Mon May 3 19:15:13 PDT 2021
Cuda compilation tools, release 11.3, V11.3.109
Build cuda_11.3.r11.3/compiler.29920130_0
nict@4a43f76a9207:~$
```

1.4、GPU卡使用规则和命名查看

- 用户上线后，可通过 nvidia-smi 查看当前 GPU 资源占用情况，如下图所示：
- 针对下图 nvidia-smi 参数进行解读：
 - GPU：GPU 编号； Name：GPU 型号；
 - Persistence-M：持续模式的状态。持续模式虽然耗能大，但是在新的 GPU 应用启动时，花费的时间更少，这里显示的是 off 的状态；
 - Fan：风扇转速，从 0 到 100%之间变动；
 - Temp：温度，单位是摄氏度；
 - Perf：性能状态，从 P0 到 P12，P0 表示最大性能，P12 表示状态最小性能（即 GPU 未工作时为 P0，达到最大工作限度时为 P12）
 - Pwr:Usage/Cap：能耗；
 - Memory Usage：显存使用率；
 - Bus-Id：涉及 GPU 总线的东西，domain：bus:device.function；
 - Disp.A：Display Active，表示 GPU 的显示是否初始化；
 - Volatile GPU-Util：浮动的 GPU 利用率；
 - Uncorr. ECC：Error Correcting Code，错误检查与纠正；
 - Compute M：compute mode，计算模式。
 - 下方的 Processes 表示每个进程对 GPU 的显存使用率。
- 注：若 Volatile GPU-Util 利用率为 0，则证明该显卡为空闲，表示可以使用，或者某张卡 Volatile GPU-Util 利用率小于 85%也可以使用这张卡，建议优先使用空闲卡 进行训练。

```
Build cuda_11.3.r11.3/compiler.29920130_0
nict@4a43f76a9207:~$ nvidia-smi
Fri Feb 10 17:47:32 2023
```

+-----+ NVIDIA-SMI 515.48.07 Driver Version: 515.48.07 CUDA Version: 11.7 +-----+									
GPU		Name	Persistence-M	Bus-Id	Disp.A	Volatile		Uncorr. ECC	
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.			
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
0	Tesla T4		Off	00000000:86:00:0	Off	0		Default	
N/A	34C	P0	28W / 70W	2MiB / 15360MiB	0%			N/A	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
1	Tesla T4		Off	00000000:87:00:0	Off	0		Default	
N/A	38C	P0	27W / 70W	2MiB / 15360MiB	0%			N/A	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
2	Tesla T4		Off	00000000:AF:00:0	Off	0		Default	
N/A	36C	P0	27W / 70W	2MiB / 15360MiB	0%			N/A	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
3	Tesla T4		Off	00000000:D8:00:0	Off	0		Default	
N/A	38C	P0	28W / 70W	2MiB / 15360MiB	0%			N/A	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+									
+-----+ Processes: +-----+									
GPU	GI	CI	PID	Type	Process name	GPU Memory			
ID	ID	ID				Usage			
+-----+ No running processes found +-----+									

海思目前只提供了基于Resnet18的分类网和YOLOV2的检测网，如果开发者的参赛作品需要用到其他网络的话，需要自己进行训练环境搭建、模型训练、模型转换等工作。

2、分类网模型训练及模型转换

步骤1：数据集准备

- 在进行分类网训练之前，请您仔细阅读社区提供的分类网训练前的准备工作，如：数据集的制作、数据集的清洗、数据集的标注等，具体请参考《[5.3.2.3分类网 \(resnet\)](#)》前3小节的内容。

步骤2：数据集上传至训练服务器

- 将制作好的数据集，上传到服务器中，本文提供的垃圾分类案例的数据集已经存放在训练服务器的 `~/ai/sample/trash_classify/dataset` 目录下了。

```
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$ ls
01_Kitchen_waste_Watermelon_rind  04_Kitchen_waste_Eggplant  07_Hazardous_waste_Waste_battery  10_Hazardous_waste_Medical_gauze  13_Recyclable_waste_Toothbrush
02_Kitchen_waste_Egg_shell        05_Kitchen_waste_Scallion  08_Hazardous_waste_Expired_cosmetics  11_Recyclable_waste_Old_dolls      14_Recyclable_waste_Old_dolls
03_Kitchen_waste_Fishbone         06_Kitchen_waste_Mushroom  09_Hazardous_waste_Woundplast       12_Recyclable_waste_Old_clip
```

步骤3：检查cuda环境

- 查看是否能获取cuda，首先在训练服务器的命令行终端输入 `python`，然后输入 `import torch`，再输入 `torch.cuda.is_available()`，若显示 `true` 则表示cuda 可获取，否则检查环境是否正确，退出输入 `exit()`即可。

```
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$ python
Python 3.6.5 |Anaconda, Inc.| (default, Apr 29 2018, 16:14:56)
[GCC 7.2.0] on linux
Type "help", "copyright", "credits" or "license()" for more
>>>
>>> import torch
>>>
>>> torch.cuda.is_available()
True
>>>
>>> exit() 退出 python环境
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$
```

步骤4：修改源码和配置文件

- 在训练服务器的命令行终端，执行下面的命令，修改开源代码，需要修改的文件路径在 `~/ai/framework/miclassification/configs/_base_/datasets` 目录下，这里需替换自己的数据集实际路径，按照下图所示进行修改，包括 `mean`、`std`、`resize`、`crop`、`data_prefix` 确认修改，这个务必配置对，否则影响模型训练的效果。

```
cd ~/ai/framework/miclassification/configs/_base_/datasets

sudo chmod 777 *

vim imagenet_bs32.py
```

```
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$ cd ~/ai/framework/miclassification/configs/_base_/datasets
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/datasets$
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/datasets$ sudo chmod 777 *
[sudo] password for nict02-s32:
Sorry, try again.
[sudo] password for nict02-s32: 输入密码
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/datasets$ vim imagenet_bs32.py
```

```

# dataset settings
dataset_type = 'ImageNet'
img_norm_cfg = dict(
    mean=[127.5, 127.5, 127.5], std=[127.5, 127.5, 127.5], to_rgb=True)
train_pipeline = [
    dict(type='LoadImageFromFile'),
    # dict(type='RandomResizedCrop', size=224),
    dict(type='Resize', size=(256, 256)),
    dict(type='CenterCrop', crop_size=224),
    dict(type='RandomFlip', flip_prob=0.5, direction='horizontal'),
    dict(type='Normalize', **img_norm_cfg),
    dict(type='ImageToTensor', keys=['img']),
    dict(type='ToTensor', keys=['gt_label']),
    dict(type='Collect', keys=['img', 'gt_label'])
]
test_pipeline = [
    dict(type='LoadImageFromFile'),
    dict(type='Resize', size=(256, 256)),
    dict(type='CenterCrop', crop_size=224),
    dict(type='Normalize', **img_norm_cfg),
    dict(type='ImageToTensor', keys=['img']),
    dict(type='Collect', keys=['img'])
]
data = dict(
    samples_per_gpu=32,
    workers_per_gpu=2,
    train=dict(
        type=dataset_type,
        data_prefix='/home/nict02-s32/ai/sample/trash_classify/dataset',
        pipeline=train_pipeline),
    val=dict(
        type=dataset_type,
        data_prefix='/home/nict02-s32/ai/sample/trash_classify/dataset',
        # ann_file='data/imagenet/meta/val.txt',
        pipeline=test_pipeline),
    test=dict(
        # replace `data/val` with `data/test` for standard test
        type=dataset_type,
        data_prefix='/home/nict02-s32/ai/sample/trash_classify/dataset',
        # ann_file='data/imagenet/meta/val.txt',
        pipeline=test_pipeline))
evaluation = dict(interval=1, metric='accuracy')
~

```

需要训练的数据集的绝对路径

- 在训练服务器的命令行终端，执行下面的命令，修改

~/ai/framework/miclassification/configs/_base_/models 目录下resnet18.py，按下图修改即可：

```

cd ../models

sudo chmod 777 *

vim resnet18.py

```

```

nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/datasets$
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/datasets$ cd ../models
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/models$
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/models$ sudo chmod 777 *
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/models$ vim resnet18.py
nict02-s32@4a43f76a9207:~/ai/framework/miclassification/configs/_base_/models$

```

```
# model settings
model = dict(
    type='ImageClassifier',
    backbone=dict(
        type='ResNet',
        depth=18,
        num_stages=4,
        out_indices=(3, ),
        style='pytorch'),
    neck=dict(type='GlobalAveragePooling'),
    head=dict(
        type='LinearClsHead',
        num_classes=13, ① num_classes根据实际训练数据集的种类来填写
        in_channels=512,
        loss=dict(type='CrossEntropyLoss', loss_weight=1.0),
        topk=(1, 5), ② 当num_classes<5时, topk的第二个值=num_classes
                    当num_classes>=5时, topk的第二个值为5
    ))
```

步骤5：查看显卡资源

- 使命令 `nvidia-smi` 查看哪张显卡空闲或 Volatile GPU-Util 利用率较低，假如第0张显卡空闲，我们就进入到下一步骤。

```
Fri Feb 10 18:51:08 2023 $ nvidia-smi
```

NVIDIA-SMI 515.48.07 Driver Version: 515.48.07 CUDA Version: 11.7									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	MIG	M.	
0	Tesla T4	Off	00000000:86:00:0	Off	0%	Default	0	N/A	
N/A	35C	P0	28W / 70W	1371MiB / 15360MiB					
1	Tesla T4	Off	00000000:87:00:0	Off	0%	Default	0	N/A	
N/A	38C	P0	27W / 70W	2MiB / 15360MiB					
2	Tesla T4	Off	00000000:AF:00:0	Off	0%	Default	0	N/A	
N/A	36C	P0	27W / 70W	2MiB / 15360MiB					
3	Tesla T4	Off	00000000:D8:00:0	Off	0%	Default	0	N/A	
N/A	38C	P0	28W / 70W	2MiB / 15360MiB					

步骤6：开始训练

- 切换到mmclassification目录下，在训练服务器的命令行终端，执行下面的命令，开始进行模型的训练

```
# 进入到mmclassification目录下
cd ~/ai/framework/mmclassification/

# 开始进行模型训练
CUDA_VISIBLE_DEVICES=0 bash ./tools/dist_train.sh
configs/resnet/resnet18_b32x8_imagenet.py 1 --work-dir ./ckpt

# 对上述命令阐述如下：
# CUDA_VISIBLE_DEVICES=0 表示使用GPU编号为0的显卡进行训练
# dist_train.sh - 训练sh脚本
# configs/resnet/resnet18_b32x8_imagenet.py - 训练依赖的配置
# 1 - GPU个数
# --work-dir ./ckpt - 模型存放的路径
```



```
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/configs/_base_/datasets$ cd ~/ai/framework/mmclassification/
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification$ CUDA_VISIBLE_DEVICES=0 bash ./tools/dist_train.sh configs/resnet/resnet18_b32x8_imagenet.py 1 --work-dir ./ckpt
/usr/local/anaconda3/lib/python3.6/site-packages/torch/distributed/launch.py:186: FutureWarning: The module torch.distributed.launch is deprecated
and will be removed in future. Use torchrun.
Note that --use_env is set by default in torchrun.
If your script expects --local_rank argument to be set, please
change it to read from os.environ['LOCAL_RANK'] instead. See
https://pytorch.org/docs/stable/distributed.html#launch-utility for
further instructions
FutureWarning:
fatal: not a git repository (or any parent up to mount point /home)
Stopping at filesystem boundary (GIT_DISCOVERY_ACROSS_FILESYSTEM not set).
2023-02-10 18:50:53.205 - mmcls - INFO - Environment Info:
```

- 当开始训练之后，我们可以看到显卡0的Volatile GPU-Util 利用率就已经大于0%了，代表该显卡正在使用中。

```
~/nicu2023$ nvidia-smi
```

Fri Feb 10 18:54:42 2023

NVIDIA-SMI 515.48.07 Driver Version: 515.48.07 CUDA Version: 11.7									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr.	ECC		
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute	M.		
0	Tesla T4	Off	00000000:86:00.0	Off	27%	0	Default		
N/A	51C	P0	2515MiB / 15360MiB			N/A			
1	Tesla T4	Off	00000000:87:00.0	Off	0%	0	Default		
N/A	33C	P8	2MiB / 15360MiB			N/A			
2	Tesla T4	Off	00000000:AF:00.0	Off	0%	0	Default		
N/A	31C	P8	2MiB / 15360MiB			N/A			
3	Tesla T4	Off	00000000:D8:00.0	Off	0%	0	Default		
N/A	33C	P8	2MiB / 15360MiB			N/A			

- 注:训练时务必指定某张空闲显卡或 Volatile GPU-Util 利用率较低的显卡进行训练。
- 训练成功后，即可在 ~/ai/framework/mmclassification/ckpt 目录下，查看生成的.pth 模型，其中最后一个 epoch_100.pth是我们所需要的。

```
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 19:20 epoch_8.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:41 epoch_80.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:44 epoch_81.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:47 epoch_82.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:49 epoch_83.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:52 epoch_84.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:55 epoch_85.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 22:58 epoch_86.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:01 epoch_87.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:03 epoch_88.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:06 epoch_89.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 19:23 epoch_9.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:09 epoch_90.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:12 epoch_91.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:14 epoch_92.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:17 epoch_93.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:20 epoch_94.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:23 epoch_95.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:26 epoch_96.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:28 epoch_97.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:31 epoch_98.pth
-rw-rw-r-- 1 nict02-s32 nict02-s32 86M Feb 10 23:34 epoch_99.pth
lrwxrwxrwx 1 nict02-s32 nict02-s32 13 Feb 10 23:37 latest.pth -> epoch_100.pth
nict02-s32@4a43f76a9207:~/ai/framework/mmclassification/ckpt$
```

步骤7：模型转换

- 目前 NNIE 配套软件及工具链仅支持以 Caffe1.0 框架，使用其他框架的网络模型需要转化为Caffe 框架下的模型，具体转换过程如下。
- 在训练服务器的命令行终端，执行下面的命令，对 ~/pytorch_to_caffe_master/example/resnet_pytorch_2_caffe.py进行修改，主要需要修改的是 name, num_classes的种类，以及您pytorch模型的绝对路径，具体修改如下图所示，注意：因为 latest.pth只是epoch_100.pth的一个软连接，我们最后指定epoch_100.pth的绝对路径就好。

```
cd ~/ai/pytorch_to_caffe_master
```

```
vim example/resnet_pytorch_2_caffe.py
```

```
nict02-s32@4a43f76a9207:~$  
nict02-s32@4a43f76a9207:~$  
nict02-s32@4a43f76a9207:~$ cd ~/ai/pytorch_to_caffe_master/  
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$  
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$ vim example/resnet_pytorch_2_caffe.py
```

```
import sys  
sys.path.insert(0, '..')  
import torch  
from torch.autograd import Variable  
from torchvision.models import resnet  
import pytorch_to_caffe  
  
if __name__ == '__main__':  
    name = 'resnet18'  # 开发者所训练的数据集的种类  
    resnet18 = resnet.resnet18(num_classes=1000)  
    checkpoint = torch.load("/home/nict02-s32/ai/framework/mmcClassification/ckpt/epoch_100.pth", map_location='cpu')  
    new_sd = {}  
    for k, v in checkpoint['state_dict'].items():  
        if not k.endswith('num_batches_tracked'):  
            if k.startswith('backbone'):  
                k_new = k.split('backbone.')[1]  
            if k.startswith('head'):  
                k_new = k.split('head.')[1]  
            new_sd[k_new] = v  # 训练出来的模型的绝对路径  
  
    resnet18.load_state_dict(new_sd)  
    resnet18.eval()  
    input = torch.ones([1, 3, 224, 224])  
    # input = torch.ones([1, 3, 224, 224])  
    pytorch_to_caffe.trans_net(resnet18, input, name)  
    pytorch_to_caffe.save_prototxt('{} .prototxt'.format(name))  
    pytorch_to_caffe.save_caffemodel('{} .caffemodel'.format(name))  
~  
~
```

- 在训练服务器的命令行终端，执行下面的命令,进行模型的转换：

```
python example/resnet_pytorch_2_caffe.py
```

```
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$  
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$ python example/resnet_pytorch_2_caffe.py  
Starting Transform. This will take a while  
139672012504184:blob1 was added to blobs  
torch ops name: {ResNet(  
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)  
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
  (relu): ReLU(inplace=True)  
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)  
  (layer1): Sequential(  
    (0): BasicBlock(  
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (relu): ReLU(inplace=True)  
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    )  
    (1): BasicBlock(  
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
      (relu): ReLU(inplace=True)  
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)  
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)  
    )  
  )  
  (layer2): Sequential(  
    (0): BasicBlock(  

```

- 转换成功后，即可在pytorch_to_caffe_master目录下生成resnet18.caffemodel和resnet18.prototxt文件。


```
conv20 was added to layers
139672012537272:conv_blob20 was added to blobs
['relu_blob16']
layer4.1.bn2
batch_norm20 was added to layers
139672012537928:batch_norm_blob20 was added to blobs
['conv_blob20']
bn_scale20 was added to layers
['batch_norm_blob20']
add8 was added to layers
139672012538328:add_blob8 was added to blobs
['batch_norm_blob20', 'relu_blob15']
layer4.1.relu
relu17 was added to layers
139672012538248:relu_blob17 was added to blobs
['add_blob8']
avgpool
ave_pool1 was added to layers
139672012538168:ave_pool_blob1 was added to blobs
['relu_blob17']
fc
fc1 was added to layers
139672012538568:fc_blob1 was added to blobs
WARNING: CANNOT FOUND blob 139672012538008
Transform Completed
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$ ls
001763.jpg LICENSE.md README.md __pycache__ caffe_analyser.py funcs.py pytorch_analyser.py resnet18.caffemodel visionmaster
Caffe Pytorch __init__.py analysis example model pytorch_to_caffe.py resnet18.prototxt
nict02-s32@4a43f76a9207:~/ai/pytorch_to_caffe_master$
```

- 至此，模型转换已经完成。

步骤8: prototxt网络层适配

- 关于prototxt网络层适配的相关操作，可以直接参考社区提供的[5.3.2.3.6 prototxt网络层适配](#)小节的内容即可。

5.3.2.3.3、... 5.3.2.3.4 分... 5.3.2.3.4... 5.3.2.3.4... 5.3.2.3.5 Py... 5.3.2.3.6 p...	5.3.2.3.6 prototxt网络层适配 5.3.2.3.6.1 网络层参数对齐及layer检查 (1) prototxt网络层适配 模型prototxt网络层适配之前，请确保
---	---

步骤9: 模型量化

- 关于模型量化的相关操作，可以直接参考社区提供的 [5.3.2.3.7 模型量化](#) 小节的内容即可。

5.3.2.3.3、... 5.3.2.3.4 分... 5.3.2.3.4... 5.3.2.3.4... 5.3.2.3.5 Py... 5.3.2.3.6 pr... 5.3.2.3.6... 5.3.2.3.6... 5.3.2.3.7 ... 5.3.2.3.8、... 5.3.2.3.8...	5.3.2.3.7 模型量化 定义：量化的过程将caffe模型转换成可以在海思芯片上执行的wk文件，同时将fl... 相关nnie_mapper配置概念请阅读源码的device/soc/hisilicon/hi3516dv300/sdk_... 配置文件说明，如下图所示： 3 NNIE开发指南 3.1 NNIE介绍 3.2 Prototxt要求 3.2.1 deploy.prototxt输入层格式 3.2.2 prototxt layer描述格式
---	---

步骤10: 模型部署板端和调试

- 关于模型部署板端和调试的相关操作，可以直接参考社区提供的 [5.3.2.3.8、模型部署板端和调试](#) 小节的内容即可。

- 5.3.2.3.5 Py...
- 5.3.2.3.6 pr...
- 5.3.2.3.6...
- 5.3.2.3.6...
- 5.3.2.3.7 ...
- 5.3.2.3.8、 ...
- 5.3.2.3.8...

5.3.2.3.8、模型部署板端和调试

5.3.2.3.8.1、概述

- wk模型生成之后，需要将生成的wk文件部署到板端，
示例代码参考在device/soc/hisilicon/hi3516dv300/sdk_

示：

3、检测网模型训练及模型转换

步骤1：数据集准备

- 在进行检测网训练之前，请您仔细阅读社区提供的检测网训练前的准备工作，如：数据集的制作、数据集的清洗、数据集的标注等，具体请参考《5.3.2.4检测网》第1小节的内容。

步骤2：数据集上传至训练服务器

- 将制作好的数据集，上传到服务器中，本文提供的手势检测案例的数据集已经存放在训练服务器的
~/ai/sample/hand_detect 目录下了。

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect$  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect$ ls  
test_dataset training_dataset  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect$
```

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect$ ls  
test_dataset training_dataset  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect$  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect$ cd test_dataset/  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset$ ls  
JPEGImages labels list  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset$  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset$ cd ../training_dataset/  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset$  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset$ ls  
JPEGImages labels list  
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset$
```

- # test_dataset为测试数据
- # training_dataset为训练数据
- # JPEGImages目录下是需要进行训练的数据集图片
- # labels目录下的txt是JPEGImages目录下对应图片所标注的labels
- # list目录下的hand_train.txt文件中保存的是JPEGImages目录下所有图片的绝对路径。
- # 注意：所有的txt文件都必须是linux格式的，可以使用 dos2unix 工具进行文件格式的转换
- # 使用方法： dos2unix filename （如果还不知道如何使用，可上网咨询度娘）

- JPEGImages目录下的图片，就是需要进行训练的数据集图片

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset/JPEGImages$ ls  
Buffy_1.jpg Buffy_432.jpg Inria_27.jpg Inria_98.jpg Poselet_8.jpg VOC2007_140.jpg VOC2007_474.jpg VOC2007_807.  
Buffy_10.jpg Buffy_433.jpg Inria_270.jpg Inria_99.jpg Poselet_80.jpg VOC2007_141.jpg VOC2007_475.jpg VOC2007_808.  
Buffy_100.jpg Buffy_434.jpg Inria_271.jpg Poselet_1.jpg Poselet_81.jpg VOC2007_142.jpg VOC2007_476.jpg VOC2007_809.  
Buffy_101.jpg Buffy_435.jpg Inria_272.jpg Poselet_10.jpg Poselet_82.jpg VOC2007_143.jpg VOC2007_477.jpg VOC2007_81.  
Buffy_102.jpg Buffy_436.jpg Inria_273.jpg Poselet_100.jpg Poselet_83.jpg VOC2007_144.jpg VOC2007_478.jpg VOC2007_810.  
Buffy_103.jpg Buffy_437.jpg Inria_274.jpg Poselet_101.jpg Poselet_84.jpg VOC2007_145.jpg VOC2007_479.jpg VOC2007_811.  
Buffy_104.jpg Buffy_438.jpg Inria_275.jpg Poselet_102.jpg Poselet_85.jpg VOC2007_146.jpg VOC2007_48.jpg VOC2007_812.  
Buffy_105.jpg Buffy_439.jpg Inria_276.jpg Poselet_103.jpg Poselet_86.jpg VOC2007_147.jpg VOC2007_480.jpg VOC2007_813.  
Buffy_106.jpg Buffy_44.jpg Inria_277.jpg Poselet_104.jpg Poselet_87.jpg VOC2007_148.jpg VOC2007_481.jpg VOC2007_814.  
Buffy_107.jpg Buffy_440.jpg Inria_278.jpg Poselet_105.jpg Poselet_88.jpg VOC2007_149.jpg VOC2007_482.jpg VOC2007_815.  
Buffy_108.jpg Buffy_441.jpg Inria_279.jpg Poselet_106.jpg Poselet_89.jpg VOC2007_15.jpg VOC2007_483.jpg VOC2007_816.  
Buffy_109.jpg Buffy_442.jpg Inria_28.jpg Poselet_107.jpg Poselet_9.jpg VOC2007_150.jpg VOC2007_484.jpg VOC2007_817.  
Buffy_11.jpg Buffy_443.jpg Inria_280.jpg Poselet_108.jpg Poselet_90.jpg VOC2007_151.jpg VOC2007_485.jpg VOC2007_818.  
Buffy_110.jpg Buffy_444.jpg Inria_281.jpg Poselet_109.jpg Poselet_91.jpg VOC2007_152.jpg VOC2007_486.jpg VOC2007_819.  
Buffy_111.jpg Buffy_445.jpg Inria_282.jpg Poselet_11.jpg Poselet_92.jpg VOC2007_153.jpg VOC2007_487.jpg VOC2007_82.  
Buffy_112.jpg Buffy_446.jpg Inria_283.jpg Poselet_110.jpg Poselet_93.jpg VOC2007_154.jpg VOC2007_488.jpg VOC2007_820.  
Buffy_113.jpg Buffy_447.jpg Inria_284.jpg Poselet_111.jpg Poselet_94.jpg VOC2007_155.jpg VOC2007_489.jpg VOC2007_83.  
Buffy_114.jpg Buffy_448.jpg Inria_285.jpg Poselet_112.jpg Poselet_95.jpg VOC2007_156.jpg VOC2007_49.jpg VOC2007_84.  
Buffy_115.jpg Buffy_449.jpg Inria_286.jpg Poselet_113.jpg Poselet_96.jpg VOC2007_157.jpg VOC2007_490.jpg VOC2007_85.  
Buffy_116.jpg Buffy_45.jpg Inria_287.jpg Poselet_114.jpg Poselet_97.jpg VOC2007_158.jpg VOC2007_491.jpg VOC2007_86.  
Buffy_117.jpg Buffy_450.jpg Inria_288.jpg Poselet_115.jpg Poselet_98.jpg VOC2007_159.jpg VOC2007_492.jpg VOC2007_87.
```

- labels目录下的txt，是JPEGImages目录下对应图片所标注的labels，文件名字都是一模一样的，只是文件后缀有不一样。

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset/labels$ ls
Buffy_1.txt      Buffy_432.txt  Inria_27.txt   Inria_98.txt   Poselet_8.txt   VOC2007_140.txt  VOC2007_474.txt
Buffy_10.txt     Buffy_433.txt  Inria_270.txt  Inria_99.txt   Poselet_80.txt  VOC2007_141.txt  VOC2007_475.txt
Buffy_100.txt    Buffy_434.txt  Inria_271.txt  Inria_271.txt  Poselet_81.txt  VOC2007_142.txt  VOC2007_476.txt
Buffy_101.txt    Buffy_435.txt  Inria_272.txt  Poselet_10.txt  Poselet_82.txt  VOC2007_143.txt  VOC2007_477.txt
Buffy_102.txt    Buffy_436.txt  Inria_273.txt  Poselet_100.txt Poselet_83.txt  VOC2007_144.txt  VOC2007_478.txt
Buffy_103.txt    Buffy_437.txt  Inria_274.txt  Poselet_101.txt Poselet_84.txt  VOC2007_145.txt  VOC2007_479.txt
Buffy_104.txt    Buffy_438.txt  Inria_275.txt  Poselet_102.txt Poselet_85.txt  VOC2007_146.txt  VOC2007_480.txt
Buffy_105.txt    Buffy_439.txt  Inria_276.txt  Poselet_103.txt Poselet_86.txt  VOC2007_147.txt  VOC2007_481.txt
Buffy_106.txt    Buffy_440.txt  Inria_277.txt  Poselet_104.txt Poselet_87.txt  VOC2007_148.txt  VOC2007_482.txt
Buffy_107.txt    Buffy_441.txt  Inria_278.txt  Poselet_105.txt Poselet_88.txt  VOC2007_149.txt  VOC2007_483.txt
Buffy_108.txt    Buffy_442.txt  Inria_279.txt  Poselet_106.txt Poselet_89.txt  VOC2007_150.txt  VOC2007_484.txt
Buffy_109.txt    Buffy_443.txt  Inria_280.txt  Poselet_107.txt Poselet_90.txt  VOC2007_151.txt  VOC2007_485.txt
Buffy_110.txt    Buffy_444.txt  Inria_281.txt  Poselet_108.txt Poselet_91.txt  VOC2007_152.txt  VOC2007_486.txt
Buffy_111.txt    Buffy_445.txt  Inria_282.txt  Poselet_109.txt Poselet_92.txt  VOC2007_153.txt  VOC2007_487.txt
Buffy_112.txt    Buffy_446.txt  Inria_283.txt  Poselet_110.txt Poselet_93.txt  VOC2007_154.txt  VOC2007_488.txt
Buffy_113.txt    Buffy_447.txt  Inria_284.txt  Poselet_111.txt Poselet_94.txt  VOC2007_155.txt  VOC2007_489.txt
Buffy_114.txt    Buffy_448.txt  Inria_285.txt  Poselet_112.txt Poselet_95.txt  VOC2007_156.txt  VOC2007_490.txt
Buffy_115.txt    Buffy_449.txt  Inria_286.txt  Poselet_113.txt Poselet_96.txt  VOC2007_157.txt  VOC2007_491.txt
Buffy_116.txt    Buffy_450.txt  Inria_287.txt  Poselet_114.txt Poselet_97.txt  VOC2007_158.txt  VOC2007_492.txt
Buffy_117.txt    Buffy_451.txt  Inria_288.txt  Poselet_115.txt Poselet_98.txt  VOC2007_159.txt  VOC2007_493.txt
```

- list目录下的hand_train.txt文件中保存的是JPEGImages目录下所有图片的绝对路径，注意：如果此处的绝对路径不对，后面模型训练的时候会因为找不到训练的图片，而导致训练失败，所以请一定要把test_dataset/list/hand_test.txt和training_dataset/list/hand_train.txt中的图片绝对路径修改正确。

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$ ls
hand_test.txt
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_1.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_10.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_100.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_101.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_102.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_103.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_104.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_105.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_106.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_107.jpg
/home/nict02-s32/ai/sample/hand_detect/test_dataset/JPEGImages/VOC2007_108.jpg
```

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset/list$
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset/list$ ls
hand_train.txt
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset/list$
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/training_dataset/list$
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_1.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_10.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_100.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_101.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_102.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_103.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_104.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_105.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_106.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_107.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_108.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_109.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_110.jpg
/home/nict02-s32/ai/sample/hand_detect/training_dataset/JPEGImages/Buffy_111.jpg
```

- 注意：所有的txt文件都必须是linux格式的，可以使用 dos2unix 工具进行文件格式的转换，使用方法：dos2unix filename （如果还不知道如何使用，可上网咨询度娘）

```
sudo apt-get install dos2unix # 如果服务器没有dos2unix这个软件的话，执行这条命令，先进行安装
```

```
dos2unix hand_test.txt # 修改某个文件的文件格式
```

```
dos2unix * # 修改该目录下的所有文件的文件格式
```



```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$ dos2unix hand_test.txt
-bash: dos2unix: 未找到命令
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$ sudo apt-get install dos2unix
[sudo] password for nict02-s32:
Reading package lists ... Done
Building dependency tree
Reading state information ... Done
The following NEW packages will be installed:
  dos2unix
0 upgraded, 1 newly installed, 0 to remove and 47 not upgraded.
Need to get 351 kB of archives.
After this operation, 1267 kB of additional disk space will be used.
Get:1 https://repo.huaweicloud.com/ubuntu bionic/universe amd64 dos2unix amd64 7.3.4-3 [351 kB]
Fetched 351 kB in 0s (3546 kB/s)
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr/share/perl5/
debconf: falling back to frontend: Readline
Selecting previously unselected package dos2unix.
(Reading database ... 73722 files and directories currently installed.)
Preparing to unpack .../dos2unix_7.3.4-3_amd64.deb ...
Unpacking dos2unix (7.3.4-3) ...
Setting up dos2unix (7.3.4-3) ...
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$ dos2unix hand_test.txt
dos2unix: converting file hand_test.txt to Unix format ...
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/list$
```

dos2unix工具找不到, 则执行下面的命令进行安装

使用dos2unix工具进行格式转换

```
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/labels$
nict02-s32@4a43f76a9207:~/ai/sample/hand_detect/test_dataset/labels$ dos2unix *
dos2unix: converting file VOC2007_100.txt to Unix format ...
dos2unix: converting file VOC2007_101.txt to Unix format ...
dos2unix: converting file VOC2007_102.txt to Unix format ...
dos2unix: converting file VOC2007_103.txt to Unix format ...
dos2unix: converting file VOC2007_104.txt to Unix format ...
dos2unix: converting file VOC2007_105.txt to Unix format ...
dos2unix: converting file VOC2007_106.txt to Unix format ...
dos2unix: converting file VOC2007_107.txt to Unix format ...
dos2unix: converting file VOC2007_108.txt to Unix format ...
dos2unix: converting file VOC2007_109.txt to Unix format ...
dos2unix: converting file VOC2007_110.txt to Unix format ...
dos2unix: converting file VOC2007_111.txt to Unix format ...
dos2unix: converting file VOC2007_112.txt to Unix format ...
dos2unix: converting file VOC2007_113.txt to Unix format ...
dos2unix: converting file VOC2007_114.txt to Unix format ...
dos2unix: converting file VOC2007_115.txt to Unix format ...
dos2unix: converting file VOC2007_116.txt to Unix format ...
dos2unix: converting file VOC2007_117.txt to Unix format ...
```

步骤3: 检查cuda环境

- 查看是否能获取cuda,首先在训练服务器的命令行终端输入 python,然后输入import torch, 再输入 torch.cuda.is_available(), 若显示true 则表示cuda 可获取, 否则检查环境是否正确, 退出输入 exit()即可

```
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$ python
Python 3.6.5 |Anaconda, Inc.| (default, Apr 29 2018, 16:14:56)
[GCC 7.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
>>> import torch
>>>
>>> torch.cuda.is_available()
True
>>>
>>> exit() 退出 python环境
nict02-s32@4a43f76a9207:~/ai/sample/trash_classify/dataset$
```

步骤4: 修改源码和配置文件

- 修改配置文件hand.data中的参数, 注意: hand.data 文件在 ~/ai/framework/darknet-master 目录下

```
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ pwd
/home/nict02-s32/ai/framework/darknet-master
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ ls
LICENSE      LICENSE.gen  LICENSE.meta LICENSE.v1  README.md  cfg      data      hand.data  include
LICENSE.fuck LICENSE.gpl  LICENSE.mit  Makefile    backup     darknet  examples  hand.names libdarknet.
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ vim hand.data
```

```

classes=1
train =/home/nict02-s32/ai/sample/hand_detect/training_dataset/list/hand_train.txt
valid =/home/nict02-s32/ai/sample/hand_detect/test_dataset/list/hand_test.txt
names =/home/nict02-s32/ai/framework/darknet-master/hand.names
backup =/home/nict02-s32/ai/framework/darknet-master/backup
~

```

classes: 为指定类别数量，手部检测模型为1，其他模型需根据实际场景进行填写
 # **train**: 指定训练数据list的路径
 # **valid**: 指定测试数据list的路径
 # **names**: 指定类别的名字
 # **backup**: 指定训练后权重的保存路径

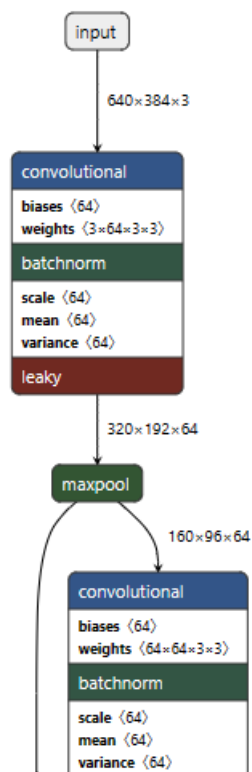
- 修改~/ai/framework/darknet-master目录下的hand.name，hand.name里面的内容，就是需要检测的类型。本文需要检测的物体只有手，因此hand.name里面的内容就只有hand，如果您需要检测多种物体，可每一行书写一种物体。

```

nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ cat hand.names
hand
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$

```

- 注意：~/ai/framework/darknet-master/cfg/下的 resnet18.cfg，可以通过 <https://netron.app/> 查看网络结构和参数，部分网络结构截图如下图所示：



步骤5：查看显卡资源

- 使命令 `nvidia-smi` 查看哪张显卡空闲或 Volatile GPU-Util 利用率较低，假如第1张显卡空闲，我们就进入到下一步骤。

```

Fri Feb 10 18:51:08 2023$ nvidia-smi
+-----+
| NVIDIA-SMI 515.48.07      Driver Version: 515.48.07      CUDA Version: 11.7     |
+-----+-----+
| GPU  Name                Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      |      Memory-Usage | GPU-Util  Compute M. |
|====+=====+===+=====+=====+=====+=====+=====+
|  0   Tesla T4              Off        | 00000000:86:00:0  Off |             0         | |
| N/A   35C    P0      28W / 70W | 00000000:86:00:0  Off | 1371MiB / 15360MiB | 0%      Default  |
|                                           1371MiB / 15360MiB |             N/A     |
+-----+-----+
|  1   Tesla T4              Off        | 00000000:87:00:0  Off |             0         | |
| N/A   38C    P0      27W / 70W | 00000000:87:00:0  Off |  2MiB / 15360MiB | 0%      Default  |
|                                           2MiB / 15360MiB |             N/A     |
+-----+-----+
|  2   Tesla T4              Off        | 00000000:AF:00:0  Off |             0         | |
| N/A   36C    P0      27W / 70W | 00000000:AF:00:0  Off |  2MiB / 15360MiB | 0%      Default  |
|                                           2MiB / 15360MiB |             N/A     |
+-----+-----+
|  3   Tesla T4              Off        | 00000000:D8:00:0  Off |             0         | |
| N/A   38C    P0      28W / 70W | 00000000:D8:00:0  Off |  2MiB / 15360MiB | 0%      Default  |
|                                           2MiB / 15360MiB |             N/A     |
+-----+-----+

```

步骤6：开始训练

- 切换到darknet-master目录下，在训练服务器的命令行终端，执行下面的命令，开始进行模型的训练

```
cd ~/ai/framework/darknet-master
```

```
mkdir backup
```

```
CUDA_VISIBLE_DEVICES=1 ./darknet detector train hand.data cfg/resnet18.cfg
```

对上述命令阐述如下：

CUDA_VISIBLE_DEVICES=1 表示使用GPU编号为1的显卡进行训练

darknet为可执行文件

detector为必选参数，直接沿用即可

train指定当前模式为训练模式，测试时候需要改成test或者valid。

hand.data是一个文件，文件中指定了训练中数据、类别、模型存储路径等信息。

resnet18.cfg是一个文件，文件中指定了训练的模型。

```

nict02-s32@4a43f76a9207:~$ cd ~/ai/framework/darknet-master
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ mkdir backup
nict02-s32@4a43f76a9207:~/ai/framework/darknet-master$ CUDA_VISIBLE_DEVICES=1 ./darknet detector train hand.data cfg/resnet18.cfg
resnet18
layer  filters  size  input              output
0 conv    64  3 x 3 / 2  640 x 384 x 3  ->  320 x 192 x 64  0.212 BFLOPs
1 max     1  2 x 2 / 2  320 x 192 x 64  ->  160 x 96 x 64
2 conv    64  3 x 3 / 1  160 x 96 x 64  ->  160 x 96 x 64  1.132 BFLOPs
3 conv    64  3 x 3 / 1  160 x 96 x 64  ->  160 x 96 x 64  1.132 BFLOPs
4 res     1  160 x 96 x 64  ->  160 x 96 x 64
5 conv    64  3 x 3 / 1  160 x 96 x 64  ->  160 x 96 x 64  1.132 BFLOPs
6 conv    64  3 x 3 / 1  160 x 96 x 64  ->  160 x 96 x 64  1.132 BFLOPs
7 res     4  160 x 96 x 64  ->  160 x 96 x 64
8 max     1  2 x 2 / 2  160 x 96 x 64  ->  80 x 48 x 64
9 conv    128 1 x 1 / 1  80 x 48 x 64   ->  80 x 48 x 128  0.063 BFLOPs
10 conv   128 3 x 3 / 1  80 x 48 x 128  ->  80 x 48 x 128  1.132 BFLOPs
11 conv   128 1 x 1 / 1  80 x 48 x 128  ->  80 x 48 x 128  0.126 BFLOPs
12 res     9  80 x 48 x 128  ->  80 x 48 x 128
13 conv   128 1 x 1 / 1  80 x 48 x 128  ->  80 x 48 x 128  0.126 BFLOPs
14 conv   128 3 x 3 / 1  80 x 48 x 128  ->  80 x 48 x 128  1.132 BFLOPs
15 conv   128 1 x 1 / 1  80 x 48 x 128  ->  80 x 48 x 128  0.126 BFLOPs
16 res    13  80 x 48 x 128  ->  80 x 48 x 128
17 max     1  2 x 2 / 2  80 x 48 x 128  ->  40 x 24 x 128
18 conv   256 1 x 1 / 1  40 x 24 x 128  ->  40 x 24 x 256  0.063 BFLOPs
19 conv   256 3 x 3 / 1  40 x 24 x 256  ->  40 x 24 x 256  1.132 BFLOPs

```

- 训练过程如下图所示：


```

Loaded: 0.000038 seconds
Region Avg IOU: 0.886993, Class: 1.000000, Obj: 0.858286, No Obj: 0.002914, Avg Recall: 1.000000, count: 12
Region Avg IOU: 0.832719, Class: 0.999996, Obj: 0.854385, No Obj: 0.002407, Avg Recall: 1.000000, count: 10
Region Avg IOU: 0.828295, Class: 1.000000, Obj: 0.823431, No Obj: 0.003020, Avg Recall: 1.000000, count: 14
Region Avg IOU: 0.767622, Class: 0.999999, Obj: 0.747484, No Obj: 0.001963, Avg Recall: 1.000000, count: 9
Region Avg IOU: 0.783933, Class: 1.000000, Obj: 0.679804, No Obj: 0.002955, Avg Recall: 1.000000, count: 14
Region Avg IOU: 0.821345, Class: 0.999999, Obj: 0.825026, No Obj: 0.002427, Avg Recall: 1.000000, count: 8
Region Avg IOU: 0.823367, Class: 0.923077, Obj: 0.831396, No Obj: 0.002912, Avg Recall: 1.000000, count: 13
Region Avg IOU: 0.763027, Class: 0.999998, Obj: 0.745521, No Obj: 0.003229, Avg Recall: 0.937500, count: 16
165648: 1.626298, 1.757994 avg, 0.001000 rate, 0.348999 seconds, 7951104 images
Loaded: 0.452634 seconds
Region Avg IOU: 0.808926, Class: 1.000000, Obj: 0.687957, No Obj: 0.002875, Avg Recall: 1.000000, count: 17
Region Avg IOU: 0.806398, Class: 1.000000, Obj: 0.748294, No Obj: 0.003240, Avg Recall: 1.000000, count: 15
Region Avg IOU: 0.666447, Class: 0.937500, Obj: 0.678420, No Obj: 0.002468, Avg Recall: 0.812500, count: 16
Region Avg IOU: 0.795317, Class: 0.999999, Obj: 0.755479, No Obj: 0.002694, Avg Recall: 0.846154, count: 13
Region Avg IOU: 0.646040, Class: 0.999997, Obj: 0.585359, No Obj: 0.003021, Avg Recall: 0.708333, count: 24
Region Avg IOU: 0.771670, Class: 0.999845, Obj: 0.727320, No Obj: 0.002294, Avg Recall: 1.000000, count: 11
Region Avg IOU: 0.861634, Class: 1.000000, Obj: 0.851164, No Obj: 0.003151, Avg Recall: 1.000000, count: 13
Region Avg IOU: 0.759751, Class: 1.000000, Obj: 0.729890, No Obj: 0.003590, Avg Recall: 0.950000, count: 20
165649: 2.393668, 1.821562 avg, 0.001000 rate, 0.345652 seconds, 7951152 images
Loaded: 0.306501 seconds
Region Avg IOU: 0.733155, Class: 1.000000, Obj: 0.751796, No Obj: 0.003858, Avg Recall: 0.882353, count: 17
Region Avg IOU: 0.602319, Class: 0.928571, Obj: 0.637364, No Obj: 0.002698, Avg Recall: 0.642857, count: 14
Region Avg IOU: 0.842518, Class: 0.937500, Obj: 0.852856, No Obj: 0.003894, Avg Recall: 1.000000, count: 16
Region Avg IOU: 0.702595, Class: 0.888887, Obj: 0.663856, No Obj: 0.003068, Avg Recall: 0.777778, count: 18
Region Avg IOU: 0.592232, Class: 0.849999, Obj: 0.551057, No Obj: 0.002706, Avg Recall: 0.750000, count: 20
Region Avg IOU: 0.818521, Class: 1.000000, Obj: 0.752892, No Obj: 0.002339, Avg Recall: 1.000000, count: 9
Region Avg IOU: 0.739972, Class: 1.000000, Obj: 0.744841, No Obj: 0.001861, Avg Recall: 1.000000, count: 9
Region Avg IOU: 0.822828, Class: 0.999999, Obj: 0.798817, No Obj: 0.003436, Avg Recall: 1.000000, count: 16
165650: 1.803199, 1.819725 avg, 0.001000 rate, 0.343814 seconds, 7951200 images
Loaded: 0.224971 seconds
Region Avg IOU: 0.767470, Class: 0.999999, Obj: 0.764907, No Obj: 0.003134, Avg Recall: 0.928571, count: 14
Region Avg IOU: 0.775499, Class: 0.923077, Obj: 0.761286, No Obj: 0.003453, Avg Recall: 0.846154, count: 13
Region Avg IOU: 0.747874, Class: 0.923077, Obj: 0.709428, No Obj: 0.002432, Avg Recall: 0.923077, count: 13
Region Avg IOU: 0.511091, Class: 0.882353, Obj: 0.550108, No Obj: 0.002715, Avg Recall: 0.588235, count: 17
Region Avg IOU: 0.775522, Class: 1.000000, Obj: 0.751023, No Obj: 0.002379, Avg Recall: 1.000000, count: 12
Region Avg IOU: 0.695097, Class: 0.941176, Obj: 0.708488, No Obj: 0.002882, Avg Recall: 0.823529, count: 17
Region Avg IOU: 0.709576, Class: 0.928571, Obj: 0.717968, No Obj: 0.002610, Avg Recall: 0.857143, count: 14
Region Avg IOU: 0.838552, Class: 1.000000, Obj: 0.789735, No Obj: 0.004420, Avg Recall: 1.000000, count: 18
165651: 2.080095, 1.845762 avg, 0.001000 rate, 0.325313 seconds, 7951248 images
Loaded: 0.022999 seconds

```

- 当开始训练之后，我们可以看到显卡0的Volatile GPU-Util 利用率就已经大于0%了，代表该显卡正在使用中。

```
nict02-s3204a43f76a9207:~$ nvidia-smi
```

```
Mon Feb 13 16:33:10 2023
```

NVIDIA-SMI 515.48.07 Driver Version: 515.48.07 CUDA Version: 11.7									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC			
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.			
						MIG M.			
0	Tesla T4	Off	00000000:86:00:0	Off	0%	0			
N/A	32C	P0	28W / 70W	2MiB / 15360MiB		Default			
						N/A			
1	Tesla T4	Off	00000000:87:00:0	Off	93%	0			
N/A	58C	P0	70W / 70W	2243MiB / 15360MiB		Default			
						N/A			
2	Tesla T4	Off	00000000:AF:00:0	Off	0%	0			
N/A	33C	P0	27W / 70W	2MiB / 15360MiB		Default			
						N/A			
3	Tesla T4	Off	00000000:D8:00:0	Off	0%	0			
N/A	35C	P0	27W / 70W	2MiB / 15360MiB		Default			
						N/A			

Processes:						
GPU	GI	CI	PID	Type	Process name	GPU Memory Usage
ID	ID	ID				

- 注:训练时务必指定某张空闲显卡或 Volatile GPU-Util 利用率较低的显卡进行训练。
- 对终端输出的一个截图做如下解释:
 - Region Avg IOU: 表示当前的subdivision内图片的平均IOU，代表预测的矩形框和真实目标的交集与并集之比，若为100%，表示我们已经拥有了完美的检测，即我们的矩形框跟目标完美重合，若为其他值较小值，表示这个模型需要进一步训练。

$$IoU = \frac{\text{Area of Overlap}}{\text{Area of Union}}$$



- Class: 标注物体分类的正确率，期望该值趋近于1。
- Obj: 越接近1越好。
- No Obj: 期望该值越来越小，但不为零。
- Avg Recall: 在recall/count中定义的，是当前模型在所有subdivision图片中检测出的正样本与实际的正样本的比值。
- count: count后的值是所有的当前subdivision图片中包含正样本的图片的数量。

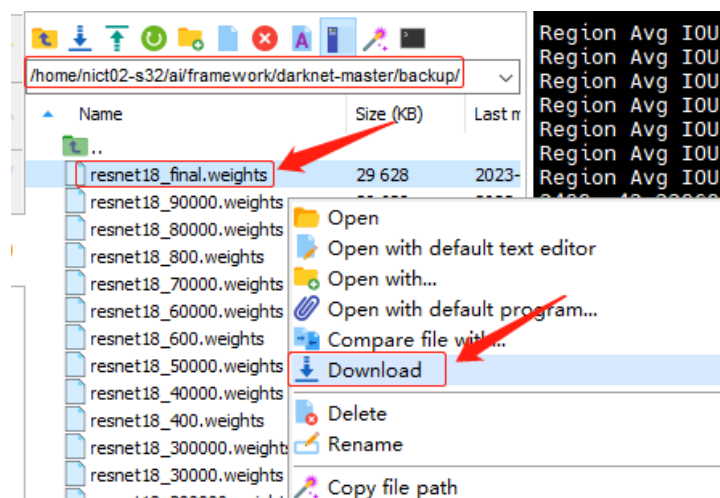
由于检测网训练时间较长，请耐心等待。

- 训练成功后，即可在~/ai/framework/darknet-master/backup 目录下查看.weights文件，若出现 resnet18_final.weights表示最终训练完成，如下图所示：

```
resnet18.backup      resnet18_140000.weights  resnet18_200000.weights  resnet18_260000.weights  resnet18_40000.weights  resnet18_70000.weights
resnet18_100.weights  resnet18_150000.weights  resnet18_210000.weights  resnet18_280000.weights  resnet18_40000.weights  resnet18_80000.weights
resnet18_10000.weights  resnet18_160000.weights  resnet18_220000.weights  resnet18_290000.weights  resnet18_50000.weights  resnet18_90000.weights
resnet18_100000.weights  resnet18_170000.weights  resnet18_230000.weights  resnet18_300000.weights  resnet18_60000.weights  resnet18_90000.weights
resnet18_110000.weights  resnet18_180000.weights  resnet18_240000.weights  resnet18_300000.weights  resnet18_60000.weights  resnet18_90000.weights
resnet18_120000.weights  resnet18_190000.weights  resnet18_250000.weights  resnet18_300000.weights  resnet18_70000.weights  resnet18_final.weights
resnet18_130000.weights  resnet18_200000.weights  resnet18_250000.weights  resnet18_300000.weights  resnet18_70000.weights  resnet18_final.weights
```

步骤7：模型转换

- 从训练服务器的~/ai/framework/darknet-master/backup 目录下，下载训练好的模型文件 resnet18_final.weights，到windows终端。



- 步骤2：将刚下载到windows的resnet18_final.weights模型文件，通过samba映射的方式上传到你 Ubuntu docker的/root/darknet2caffe/目录下

```
root@hispark:~/darknet2caffe# ls
README.md  __pycache__  caffe_layers  cfg  cfg.py  darknet2caffe.py  prototxt  prototxt.py  resnet18.cfg  resnet18_final.weights  weights
root@hispark:~/darknet2caffe#
```

- 步骤3: 执行下面的命令, 将darknet模型转换为caffe模型。

```
# 转换命令遵循:
# python cfg[in] weights[in] prototxt[out] caffemodel[out]
# 本文转换命令如下:
python3.8 darknet2caffe.py resnet18.cfg resnet18_final.weights
resnet18.prototxt resnet18.caffemodel
```

如下图所示:

```
root@hispark:~/darknet2caffe# python3.8 darknet2caffe.py resnet18.cfg resnet18_final.weights resnet18.prototxt resnet18.caffemodel
WARNING: Logging before InitGoogleLogging() is written to STDERR
I0118 20:21:26.820554 81 upgrade_proto.cpp:69] Attempting to upgrade input file specified using deprecated input fields: resnet18.prototxt
I0118 20:21:26.820636 81 upgrade_proto.cpp:72] Successfully upgraded file specified using deprecated input fields.
I0118 20:21:26.820643 81 upgrade_proto.cpp:74] Note that future Caffe releases will only support input layers and not input fields.
I0118 20:21:26.821120 81 net.cpp:53] Initializing net from parameters:
name: "Darknet2Caffe"
state {
  phase: TEST
  level: 0
}
layer {
  name: "input"
  type: "Input"
  top: "data"
  input_param {
    shape {
      dim: 1
      dim: 1
    }
  }
}
```

- 当转换成功后, 会在darknet2caffe目录生成一个resnet18.caffemodel和一个resnet18.prototxt文件, 如下图所示。

```
I0118 20:21:26.984241 81 net.cpp:244] This network produces output layer36-conv
I0118 20:21:26.984344 81 net.cpp:257] Network initialization done.
save prototxt to resnet18.prototxt
save caffemodel to resnet18.caffemodel
root@hispark:~/darknet2caffe# ls
README.md  _pycache_  caffe_layers  cfg.py  prototxt  resnet18.caffemodel  resnet18.prototxt  weights
_pycache_  cfg        darknet2caffe.py  prototxt.py  resnet18.cfg  resnet18_final.weights
```

电脑 > docker (\\192.168.56.106) (Z:) > root > darknet2caffe

名称	修改日期	类型	大小
.git	2022/5/4 17:09	文件夹	
pycache	2022/5/4 18:32	文件夹	
caffe_layers	2022/5/4 17:09	文件夹	
cfg	2022/5/4 17:09	文件夹	
prototxt	2022/5/4 17:09	文件夹	
weights	2022/5/4 17:09	文件夹	
cfg.py	2022/5/4 17:09	Python 源文件	11 KB
darknet2caffe.py	2022/5/4 18:21	Python 源文件	22 KB
prototxt.py	2022/5/4 18:22	Python 源文件	7 KB
README.md	2022/5/4 17:09	MD 文件	3 KB
resnet18.cfg	2022/12/8 9:26	Configuration 源...	4 KB
resnet18_final.weights	2023/1/18 18:31	WEIGHTS 文件	29,629 KB
resnet18.prototxt	2023/1/18 20:21	PROTOTXT 文件	19 KB
resnet18.caffemodel	2023/1/18 20:21	CAFFEMODEL 文...	29,639 KB

步骤8: 模型量化

- 关于模型量化的相关操作, 可以直接参考社区提供的 [5.3.2.4.4 模型量化](#) 小节的内容即可。

- 5.3.2.3.8、模型部署板端...
 - 5.3.2.3.8.1、概述
 - 5.3.2.3.8.2、编译
 - 5.3.2.3.8.3、拷贝可执...
 - 5.3.2.3.8.4、拷贝mnt...
 - 5.3.2.3.8.5、功能验证
- 5.3.2.4、检测网
 - 5.3.2.4.1 数据集制作和标注
 - 5.3.2.4.2 本地模型训练
 - 5.3.2.4.3 Darknet模型转...
 - 5.3.2.4.4 模型量化
 - 5.3.2.4.5、模型部署板端...
 - 5.3.2.4.5.1、概述
 - 5.3.2.4.5.2、编译
 - 5.3.2.4.5.3、拷贝可执...
 - 5.3.2.4.5.4、拷贝mnt...

5.3.2.4.4 模型量化

注：nnie_mapper配置概念请阅读源码的device/soc/hisilicon/hi3516dv300/sdk_linux/sample/d南.pdf》3.5.2章节配置文件说明

- 由于yolo2网络的最后一层需要通过AI CPU推理，转换之前，需要手工将其删除，如下图所示

```
layer {
  bottom: "layer36-conv"
  top: "layer36-conv"
  name: "layer36-scale"
  type: "Scale"
  scale_param {
    bias_term: true
  }
}
layer {
  bottom: "layer36-conv"
  top: "layer37-region"
  name: "layer37-region"
```

步骤9：模型部署板端和调试

- 关于模型量化的相关操作，可以直接参考社区提供的 [5.3.2.4.5、模型部署板端和调试](#) 小节的内容即可。

- 5.3.2.3.8.4、拷贝mnt...
- 5.3.2.3.8.5、功能验证
- 5.3.2.4、检测网
 - 5.3.2.4.1 数据集制作和标注
 - 5.3.2.4.2 本地模型训练
 - 5.3.2.4.3 Darknet模型转c...
 - 5.3.2.4.4 模型量化
 - 5.3.2.4.5、模型部署板端...
 - 5.3.2.4.5.1、概述
 - 5.3.2.4.5.2、编译
 - 5.3.2.4.5.3、拷贝可执...
 - 5.3.2.4.5.4、拷贝mnt...

5.3.2.4.5、模型部署板端和调试

5.3.2.4.5.1、概述

- wk模型生成之后，需要将生成的wk文件部署到板端，本文基于SDK sample的yolov2测试，示例代码参考在device/soc/hisilicon/hi3516dv300/sdk_linux/sample/platform/sv/sample_nnie.c，如下图所示：

```
BUILD.gn  sample_nnie.c x
device > soc > hisilicon > hi3516dv300 > sdk_linux > sample > platform > sv > nnie > sample > C samp
2860
2861 /*.function::show_YOLOV2_sample(image:416x416-U8_C3):*/
2862 void SAMPLE_SVP_NNIE_Yolov2(void)
2863 {
2864     const HT_CHAR *pSrcFile = "/data/nnie_image/rgb_planar/street_cars"
```